The configuration might include resource handles to backing services like a database or a messaging system, credentials to access third-party APIs, and feature flags. Ask yourself if any credential or environment-specific information would be compromised should your codebase suddenly become public. That will tell you whether you have correctly externalized the configuration.

To be compliant with this factor, the configuration can't be included in the code or tracked in the same codebase. The only exception is the default configuration, which can be packaged with the application codebase. You can still use configuration files for any other type of configuration, but you should store them in a separate repository.

The methodology recommends storing configuration as environment variables. By doing so, you can have the same application deployed in different environments but with different behaviors depending on the environment's configuration.

2.1.6 Logs

A cloud native application isn't concerned with routing and storage of logs. Applications should log to the standard output, treating logs as events emitted in a sequence ordered by time. Log storage and rotation are not application responsibilities anymore. An external tool (a *log aggregator*) will fetch, collect, and make logs available for inspection.

2.1.7 Disposability

In a traditional environment, you would take much care of your applications, ensuring they stay up and running and never terminate. In a cloud environment, you don't need to care that much: applications are ephemeral.